Türkçe E-Ticaret Aramasında Üç Sınıflı Relevance

Gold irrelevant verisi olmadan, tek GPU üzerinde LLM judge ile sınıflandırma

DREAMTEAM

Trendyol E-Ticaret Yarışması 2026 · Stage 2 Teknik Rapor

> **Sonuç: 90.000 satırlık private test formatında macro-F1 0,9131, tek NVIDIA A100-SXM4-40GB üzerinde 46,9 dakika.** Organizatör limiti 150 dakikadır. Karşılaştırma için oracle olarak optimize edilmiş lexical baseline 0,4335'te kalmaktadır.

ÖZET

Trendyol Stage-2 yarışmasında (query, product) çiftlerini relevant, semi-relevant ve irrelevant olarak etiketleme problemini modelledik. Organizatör 500.000 satırlık eğitim verisinde hiç irrelevant satır vermedi. Gerçek irrelevant örneklerin rastgele kategori dışı ürünler olmadığını gördük. Bu ürünler query ile sözcüksel olarak yakın olan, ancak kilit bir niteliği ihlal eden zorlu negatiflerden oluşuyordu. Bu veri yapısı nedeniyle optimize edilmiş lexical logistic regression taban çizgisi macro-F1 0,4335 seviyesinde kaldı. Farklı encoder tabanlı stratejileri denedik, ardından büyük dil modellerine geçtik.

Sistemimizi Gemma-4-31B-it INT8 modelini temel alan bir LLM judge olarak tasarladık. Çıkarım maliyetini düşürmek için metin üretiminden tamamen vazgeçtik ve forced-token read-out mekanizmasını uyguladık. Modelin sadece prefill aşamasını çalıştırdık. Asistan yanıtını spesifik bir formatla başlattık ve her etiketi temsil eden vocabulary token'larının logprob marjinallerini doğrudan okuduk. Sınıf hedeflerini yumuşatmak ve çıktı dağılımını ayarlamak için IPF read-out yöntemini test ettik.

Nihai champion modelimiz 90.000 satırlık query-disjoint private test setinde macro-F1 0,9131 skorunu elde etti. Tek bir A100-SXM4-40GB donanımında gelişmiş tokenisation ve paylaşımlı prefix cache tekniklerini kullandık. Toplam çıkarım süresini 46,9 dakika olarak ölçtük. Çıkarım işlemini 150 dakikalık yarışma limitinin altında tamamlayarak katı donanım kısıtlarını başarıyla yönettik.

1. Problem ve Zorluk

Trendyol Stage-2 yarışmasındaki temel problem, verilen bir (query, product) çiftini relevant, semi-relevant veya irrelevant olarak sınıflandırmaktır. Modelin başarısını macro-F1 metriği üzerinden ölçtük. Organizatör tarafından sağlanan 500.000 satırlık eğitim verisi yalnızca 250.000 relevant ve 250.000 semi-relevant örnekten oluşuyordu. Çözülmesi gereken asıl zorluk tam olarak burada ortaya çıkıyor. Elimizde tek bir gold irrelevant örneği yoktu.

Test setini incelediğimizde irrelevant etiketinin rastgele alakasız bir ürün anlamına gelmediğini gördük. Gerçek irrelevant örnekleri query ile aynı üst kategoriden geliyor ve lexical olarak oldukça yakın duruyor. Sadece kilit bir niteliği ihlal ediyorlar. Dört somut örnek bu durumu netleştiriyor: "yeşil topuklu ayakkabı" sorgusunda yeşil sneaker, "penti plaj çantası" sorgusunda bikini altı, "bershka şort" sorgusunda geniş yaka

top ve "beyaz şık elbise" sorgusunda çiçek desenli balon kol elbise irrelevant olarak etiketlenmiş. Rastgele veya kategori dışı negatif örneklerle eğitilen bir model bu ince farkı göremiyor. Model bu ürünleri doğrudan semi-relevant sanıyor.

Bu zorluğu sayısallaştırmak için bir lexical logistic regression baseline modeli kurduk. Model overlap, head-noun ve brand feature'ları kullanıyordu. Eşikleri public_solution üzerinde ORACLE olarak optimize etmemize rağmen macro-F1 skoru 0,4335 seviyesinde takıldı. Rastgele tahminin 0,333 olduğu bir senaryoda bu skor oldukça zayıf. Sınıf bazında F1 metriklerine baktığımızda problemin yapısı ortaya çıkıyor. Skorları relevant için 0,481, semi-relevant için 0,399 ve irrelevant için 0,387 olarak ölçtük. Bu sayılar bize relevant ve semi-relevant ekseninin irrelevant sınıfını hiçbir şekilde ayırmadığını gösteriyor. Başarılı bir sınıflandırma için tamamen ayrı bir sinyal ekseni gerekiyor.

2. Veri ve EDA Bulguları

Yarışmanın ana problemi (query, product) çiftlerini relevant, semi-relevant veya irrelevant olarak sınıflandırmaktır ve hedef metrik macro-F1 olarak belirlenmiştir. Organizatör tarafından sağlanan train.csv dosyası 500.000 satırdan oluşur. Bu verinin 250.000 adedi relevant ve geri kalan 250.000 adedi semi-relevant etiketine sahiptir. Organizatör eğitim setinde hiç irrelevant satırı vermemektedir. public_test.csv dosyası 7.500 etiketsiz satır barındırır. public_solution.csv ise her sınıftan 2.500 örnek içerecek şekilde dengelidir. Nihai değerlendirmenin yapıldığı private test verisi 90.000 satırdır ve 30.000 relevant, 30.000 semi-relevant, 30.000 irrelevant dağılımıyla tamamen dengelidir. Veri setleri arasındaki en katı kural query-disjoint yapıdır. Eğitim, public ve private aşamalarındaki sorgular birbirleriyle kesinlikle kesişmez.

Yan veri kaynaklarını incelediğimizde items.csv dosyasında 962.873 ürün ve terms.csv dosyasında 50.153 sorgu bulunduğunu gördük. Bu yüksek sayılara rağmen katalog verisinin faydası oldukça sınırlı kalmıştır. Yaptığımız ölçümlerde test başlıklarının yalnızca %16 gibi bir kısmının items.csv içinde yer aldığını tespit ettik. Kapsama oranının bu kadar düşük olması dış kaynaklı ürün bilgilerine dayalı veri zenginleştirme adımlarını boşa çıkarmıştır. Modelin doğrudan test satırlarındaki salt metin üzerinden çalışması zorunlu hale gelmiştir.

Eğitim verisindeki irrelevant eksikliğini çözmek için Stage-1 aşamasından kalan dosyaları inceledik ve kullandık. training_pairs_stage1.csv dosyasında tamamı label=1 olan 250.000 çift yer alıyordu ve submission_pairs_stage1.csv toplam 3.359.679 aday çift içeriyordu. Verileri karşılaştırdığımızda Stage-1 içindeki label=1 satırlarının, Stage-2 setindeki relevant satırlarıyla neredeyse birebir örtüştüğünü ölçtük (250.000 örneğin 249.984 adedi eşleşti). Bu örtüşme bizim için çok faydalı bir fırsat yarattı. Stage-1 negatifleri bize doğrudan geniş bir {semi-relevant ∪ irrelevant} havuzu verdi. Bu havuz sayesinde eksik olan üçüncü sınıfın dağılımını doldurduk ve eğitimin başlarında modelin sınırları daha iyi öğrenmesini sağladık.

3. Negative Sampling ve Veri Üretimi

Eğitim setinde organizatör tarafından sağlanan 250.000 relevant ve 250.000 semi-relevant örnek bulunuyordu. Veri setinde hiç irrelevant satır yoktu. Bu eksikliği gidermek için on farklı negative mining stratejisi tasarladık ve test ettik. Kontrol grubu olarak A1 random-crosscat yöntemini belirledik. Hiyerarşik yakınlık için A2 sibling-leaf, metinsel benzerlik üzerinden A3 bm25-nearmiss ve vektör uzayında A4 dense-hardneg stratejilerini denedik. Nitelik uyumsuzluklarına odaklanan A5 attr-contradiction, önceki aşamadan gelen A6 stage1-PU-mining ve A7 llm-judge-distill yöntemlerini kurguladık. Son olarak A8 query-swap, A9

ordinal-selftrain ve A10 hybrid-curriculum ile listeyi tamamladık. Tüm bu stratejileri adil bir şekilde karşılaştırmak için sabit bir reference trainer kullandık. Bu aşamada dbmdz/convbert-base-turkish-cased modelini 3 sınıf, seq 160, batch 64, learning rate 3e-5, 1 epoch ve seed 0 ayarlarıyla eğittik.

Yarışmanın public_solution veri setini incelediğimizde kritik bir gözlem yaptık. Gerçek irrelevant örnekleri rastgele seçilmiş alakasız ürünlerden oluşmuyordu. Aksine, query ile aynı üst kategoriye ait ve lexical olarak yakın ürünlerdi. Ancak bu ürünler kilit bir niteliği ihlal ediyordu. Örneğin "yeşil topuklu ayakkabı" aramasına yeşil sneaker, "penti plaj çantası" aramasına bikini altı, "bershka şort" aramasına geniş yaka top veya "beyaz şık elbise" aramasına çiçek desenli balon kol elbise eşleşmeleri irrelevant olarak etiketlenmişti. Rastgele veya kategori dışı negatiflerle eğitilen modeller bu örnekleri semi-relevant sanıyordu. Tasarladığımız A5 attr-contradiction stratejisi tam olarak bu dinamiği yakaladı. Gözlemlenen irrelevant semantiğine en çok uyan yöntem buydu, çünkü doğrudan arama terimi ile ürün arasındaki spesifik nitelik çelişkilerini üretiyordu.

Taban çizgisi deneylerimiz, relevant ve semi-relevant ekseninin irrelevant sınıfını ayırmadığını gösterdi. Lexical logistic regression modelinde sınıflar arası skorlar relevant için 0,481, semi-relevant için 0,399 ve irrelevant için 0,387 olarak ölçüldü. Ayrı bir sinyal eksenine ihtiyacımız vardı. Birinci dönem encoder ve negative mining çalışmalarımız sonucunda ulaşabildiğimiz en iyi macro-F1 skoru 0,7369 oldu. Bu noktada encoder mimarilerinin kapasite sınırına ulaştığımızı gördük. Modeli daha fazla sentetik veriyle zorlamak yerine yaklaşımımızı tamamen değiştirdik. Geleneksel sınıflandırma mimarilerinden vazgeçtik ve doğrudan LLM judge ile forced-token read-out mekaniğine geçiş yaptık.

4. Model Seçimi

Yarışma için ana çözüm olarak lokeshe09/gemma-4-31B-it-INT8 modelini seçtik. Bu karar basit bir quantisation tercihi değil, tamamen donanım kısıtlarının dikte ettiği bir zorunluluktu. Hedef ortamdaki A100-SXM4-40GB kartları SM80 mimarisini kullanıyor. SM80 mimarisinde INT8 W8A8 veri tipi tensor core üzerinde native olarak işleniyor. FP8 formatı için ise durum farklı. SM80 üzerinde FP8 tensor core bulunmuyor. Bu donanım eksikliği yüzünden sisteme yüklenen herhangi bir FP8 checkpoint doğrudan çalıştırılamıyor ve zorunlu olarak Marlin W8A16 formatına demote oluyor. Dönüşüm işlemi modeli yavaşlatıyor.

Alternatif bir strateji olarak RedHatAI/gemma-4-26B-A4B-it-FP8-dynamic checkpoint'ini ucuz bir MoE gate olarak kullanmayı denedik. Planımız tüm satırları önce bu gate ile puanlamak ve yalnızca en belirsiz örnekleri champion modele yükselten bir cascade kurmaktı. Ölçümlerimiz bu yapının donanım gerçeklerine çarptığını gösterdi. Gate modelinin FP8 formatında olması, aynı Marlin W8A16 demotion sürecini tetikledi ve modeli yavaşlattı. Cascade mimarisinin toplam duvar saati süresinin yüzde 62 ile yüzde 80 arasındaki kısmı yalnızca gate modelinin puanlama işlemine harcandı. Gate'in bu donanım uyumsuzluğu yüzünden beklenen hızı verememesi üzerine karmaşık yapıdan vazgeçtik. Tüm yükü tensor core üzerinde native hızda koşan tekil INT8 champion modele devrettik.

5. Model Mimarisi ve Read-out

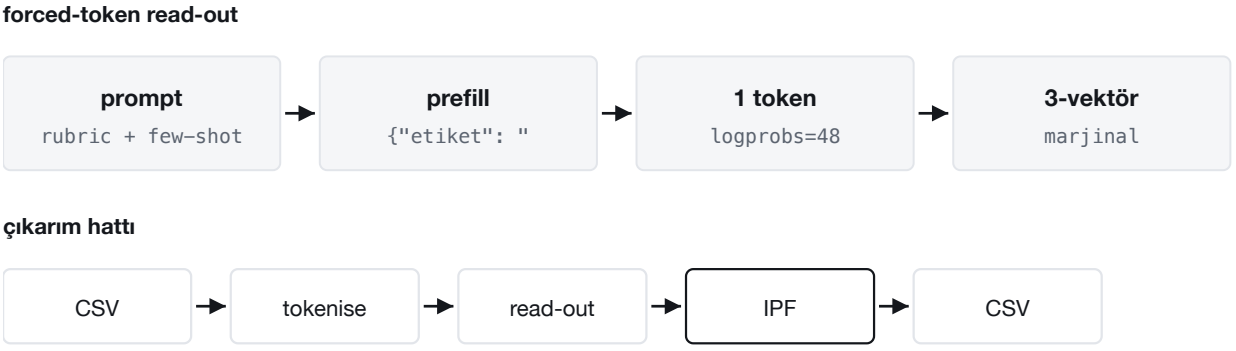
Geleneksel metin üretimi yerine, modeli sınıflandırma modunda çalıştırmak için forced-token read-out mekanizmasını uyguladık. Bu yöntemde, modelin assistant turn kısmını doğrudan **{"etiket":** " dizgisiyle prefill ettik. Modelden serbest bir metin üretmesini istemedik. Bunun yerine sadece tam bir token decode

ettik. Bu işlem için max_tokens=1 ve logprobs=48 parametrelerini kullandık. Çıktı olarak elde ettiğimiz 3-vektör, her bir etiketi tekil olarak prefix'leyen vocabulary token'larının marjinalidir.

Bu tasarım sayesinde hiçbir şey generate edilmedi. Çıktının sonunda parse edilecek bir JSON formatı beklemedik. Üretim aşamasının tamamen atlanması, malformed çıktı riskini sıfıra indirdi. Bu mekanizmanın doğruluğunu teyit etmek için label_mass_mean değerini ölçtük. Tüm koşumlarda bu değer 0,998 ile 1,000 arasında gerçekleşti, ayrıca tam 1,0 değerini de gözlemledik. Bu yüksek olasılık kütlesi, read position noktasının kesinlikle doğru yerde olduğunu kanıtlayan bir invariant olarak çalıştı.

Prompt yapısını iki ana bileşen üzerine kurduk: champ3 rubric ve küresel bir few-shot bloğu. Söz konusu few-shot bloğunu train.parquet dosyasından seed 4 kullanarak deterministik olarak çektik. Sadece bu örneklem bloğu 8.710 karakterlik bir hacim kapladı. Champion modeline verilen system message toplamda 14.189 karaktere ulaştı. Bu geniş metin, modelin görev sınırlarını ve sınıflandırma kurallarını katı bir şekilde tanımladı.

Bu büyük yapı, tüm satırlar için ortak bir paylaşılan bağlam oluşturdu. Ölçümlerimize göre bu paylaşılan metin 4.951 ile 4.957 token arasında bir prefix üretti. Satır gövdesi ise ortalama 145 token uzunluğundaydı. Tüm sistemi bu geometri üzerine inşa ettik ve her bir satırın sonuna geldiğimizde doğrudan read-out aşamasına geçtik.



Şekil 5. Üst şema, prompt ve prefill sonrası logprob ile tek token çözüp marjinal vektör üreten read-out aşamasıdır. Alt tokenisation hattında metin üretilmediğinden JSON ayrıştırma ve bozuk çıktı riski yoktur. Küresel bir adım olan IPF, tek satır yerine tüm veriyi işleyerek macro-F1 değerini optimize eder.

6. Veri Temizleme ve Input Preprocessing

Girdi işleme aşamasına standart CSV okuma hatalarını önleyerek başladık. Veriyi parse ederken quoted multiline alanları eksiksiz koruduk. Kimlik sütunu olan row_id değerini özellikle string olarak okuduk. Böylece 007 gibi değerlerin 7 formatına dönmesini engelledik.

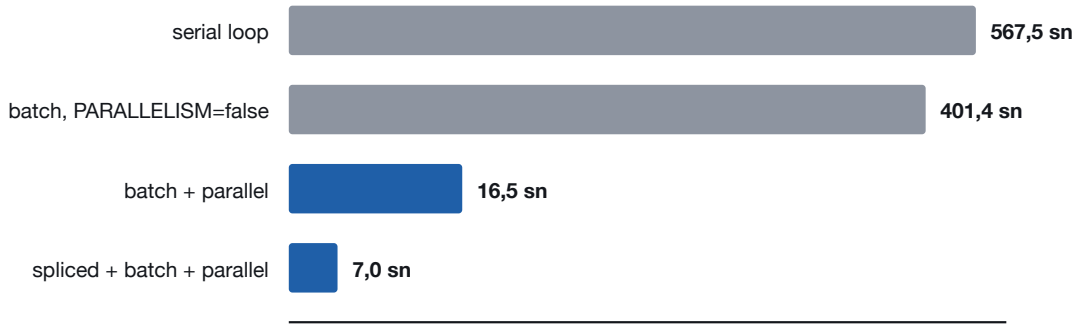
Sonrasında tokenisation optimizasyonu üzerine yoğunlaştık. Naif satır başı encode işlemi 90.000 prompt için 588,0 saniye sürüyordu. Bu süreyi düşürmek için iki temel değişiklik yaptık. İlk olarak paylaşılan prefix kısmını yalnızca bir kez tokenize ettik ve satır gövdesini sonradan splice işlemiyle ekledik. İkinci olarak TOKENIZERS_PARALLELISM ayarını yeniden açtık. Transformers kütüphanesi import aşamasında bu ayarı false değerine sabitliyor ve sistemdeki 256 thread biriminin 255 tanesi boşta kalıyordu.

Farklı yöntemlerin etkisini ölçmek için 20.000 örnek üzerinde bir ablation çalışması yaptık ve sonuçları 90.000 satıra ölçekledik.

Yöntem	Süre (sn)
serial loop	567,5
batch + PARALLELISM=false	401,4
batch + parallel	16,5
spliced + batch + parallel	7,0

Gerçek koşumda toplam tokenisation süresi 588,0 saniyeden 27,0 saniyeye indi. Bu hızlandırma 90.000 satırın tamamında bit-identical sonuç verdi. Tam olarak 0 mismatch ölçtük. Uyguladığımız encode_spliced metodu körü körüne bir güven varsaymıyor. Doğrulamak için bir örneği orijinal yoldan yeniden encode edip tam id listelerini karşılaştırıyor. Herhangi bir uyuşmazlık durumunda doğrudan eski loop yapısına düşüyor.

Alternatif olarak disk cache kullanmayı denedik ancak bu fikirden vazgeçtik. Okuma ve yeniden kurma işlemi 3,92 saniye sürerken sıfırdan yeniden hesaplama yalnızca 1,55 saniye tuttu.



Şekil 4. Logaritmik eksende sunulan bu grafik, 90.000 satıra ölçeklenmiş veri üzerinde gerçekleştirilen tokenisation ablation sonuçlarını göstermektedir. Paralel batch işlemi tek başına süreyi 16,5 saniyeye indirerek 24 kat hızlanma sağlarken, sürece prefix splice dahil edilmesi işlem süresini 7,0 saniyeye düşürerek toplamda 81,1 kat performans artışı sunmaktadır.

7. Training Strategy

Bu çözümde champion model için herhangi bir fine-tune işlemi uygulamadık. Bunun yerine hazır bir instruction-tuned checkpoint kullandık ve modeli doğrudan rubric ile few-shot block üzerinden yönlendirdik. Geleneksel ağırlık güncelleme işlemlerini sadece birinci dönemde inşa ettiğimiz encoder hattında gerçekleştirdik. Bu aşamadaki reference trainer modelimiz dbmdz/convbert-base-turkish-cased idi. Eğitim hiperparametrelerini 3 sınıf, seq 160, bs 64, lr 3e-5, 1 epoch ve seed 0 olarak sabitledik. En iyi encoder koşumunda macro-F1 değerini 0,7369 ölçtük ve bu seviyeden sonra modeli eğitmeyi bıraktık.

Modelin davranışını kalibre eden few-shot block, train.parquet dosyasından seed 4 kullanılarak deterministik bir şekilde çekildi. Toplam 8.710 karakterden oluşan bu rendered metni statik bir asset olarak SHA özetiyle birlikte teslimat paketine dahil ettik. Bu mimari tercih sayesinde inference aşamasında 500.000 satırlık orijinal eğitim dosyasına hiç ihtiyaç duymuyoruz. Tüm in-context sinyali doğrudan bu önceden hesaplanmış dosyadan okunuyor.

Sessizce değişmiş farklı bir few-shot block, sessizce farklı bir model demektir. Modelin ürettiği sonuçlar büyük oranda bu bloğun içeriğine ve sıralamasına bağlıdır. Bu yüzden eksik veya doğrulanmamış bir asset bizim için doğrudan fatal bir hatadır. Sistem başlangıçta SHA doğrulamasını geçemezse sessizce yanlış tahminler üretmek yerine çalışmayı tamamen durdurur.

8. Validation Strategy

Veri sızıntısını engellemek için public_solution içindeki 7.500 satırı seed 42 kullanarak tabakalı biçimde böldük. İlk 500 satırlık kısmı ayırdık. Bu seti yalnızca irrelevant tanımını kalibre etmek için bir ön inceleme alanı olarak kullandık. Kalan 7.000 satırlık holdout kümesini ise sadece skorlama işlemi için sakladık. Ajanlara hiçbir şekilde doğrudan etiket vermedik. Çalışma boyunca onlara yalnızca skor geri bildirimi sağladık.

Geliştirme döngüsünü güvenceye almak için bağımsız bir 90.000 satırlık değerlendirme seti kurduk. Raporladığımız nihai sayıların tamamını doğrudan orada ölçtük. 7.000 satırlık holdout üzerinde paired-bootstrap standart hatası 0,0060 seviyesindeydi. Bu durum küçük iyileştirmelerin gürültüden ayrılmasını zorlaştırıyordu. 90.000 satırlık sette paired-bootstrap standart hatasının yaklaşık 0,0017 değerine düştüğünü hesapladık. Bu sayede 7.000 satırda görünmez olan ince performans farklarını istatistiksel olarak ayırt edilebilir kıldık.

Nihai champion yapılandırmasını her iki veri kesitinde de çalıştırdık. Model 7.000 satırlık holdout üzerinde **0,9172** macro-F1 skoruna ulaştı. Aynı sistemi 90.000 satırlık büyük sete taşıdığımızda **0,9131** macro-F1 elde ettik. İki metrik arasındaki -0,004 puanlık fark sistemin küçük sete aşırı uyum sağlamadığını kanıtlıyor. Modelin yeteneği geniş veri üzerinde güvenle korunuyor.

9. Inference Strategy: Solution Routine

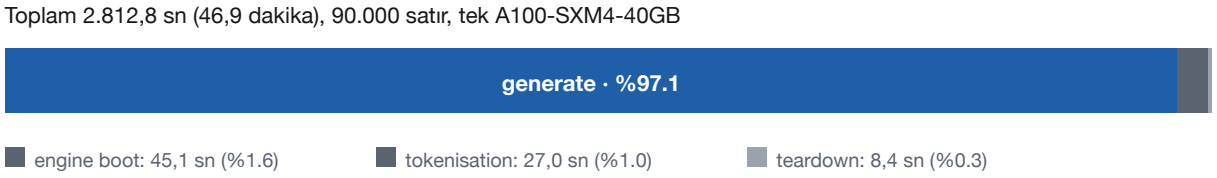
Nihai çözümümüzde karmaşık bir cascade mimarisinden vazgeçtik ve tek bir checkpoint ile çalışan yalın bir hat kurduk. Sistem herhangi bir gate veya yönlendirme adımı içermiyor. Çalışma rutini sırasıyla CSV okuma ve sözleşme doğrulaması ile başlıyor. Ardından gövde metnini sabit bir ön eke ekleyen prefix-splice tokenisation işlemini uyguluyoruz. Model, tek bir forward pass üzerinden forced-token read-out mekanizmasıyla çalışıyor. Çıktı olasılıklarını elde ettikten sonra IPF read-out adımını işleme alıyoruz. Süreç, atomik CSV yazımı ve dosyanın yeniden okunarak doğrulanmasıyla sona eriyor.

Sunum katmanındaki kararlarımızı doğrudan donanım ölçümlerine dayandırdık. Parametre olarak max_model_len değerini 5250 olarak belirledik, çünkü public_test setindeki en uzun prompt tam olarak 5178 token içeriyor. Bellek tarafında gpu_memory_utilization ayarını 0,94 seviyesinde tuttuk. Bu değeri 0,97 yaptığımızda FlashInfer için ayrılan workspace sampler bileşenine yer bırakmıyor ve warm-up sırasında OOM hatasına yol açıyor. Dikkat mekanizması için FlashInfer attention backend kullandık ve prefix cache özelliğini açık tuttuk. Ayrıca enforce_eager=True ayarını tercih ettik. Bizim read-out işlemimiz saf bir prefill adımı olduğu için eager mod %0,4 daha hızlı çalışıyor.

Sistem kaynaklarını izlerken KV havuzu ölçümünde kritik bir tuzak tespit ettik. Standart vLLM log çıktısındaki GPU KV cache size satırı yalnızca tek bir cache group sayısını raporluyor ve gerçek değeri 5,53 kat düşük gösteriyor. Bu yanıltıcı metriği aşmak için canlı bir prob ile ölçüm yaptık. Gerçek kapasitenin 35.104 token olduğunu ve 7,08 kat headroom bulunduğunu gördük. Bu oranı dört bağımsız koşumda test ettik ve hepsinde aynı sonucu ürettiğini doğruladık.

Dağıtım ortamı organizatör tarafından son derece kısıtlayıcı kurallarla çerçevelenmişti. Konteynerler read-only rootfs, dışa kapalı network none, 70 GB RAM, takassız bellek ve 8 GB tmpfs limitleriyle çalışıyor. Ayrıca süreç arbitrary uid ile yürütülüyor. Bu durum naif bir Docker imajını anında kırıyor. Örneğin vLLM, Triton veya HF önbellekleri doğrudan $HOME dizinine yazmaya çalışıyor ve kök dosya sistemi salt okunur olduğu

için başarısız oluyor. Rastgele atanan kullanıcı kimliğinin /etc/passwd dosyasında bir kaydı bulunmuyor, bu da os.path.expanduser çağrısının import aşamasında doğrudan çökmesine neden oluyor. Tüm bu kısıtları yapılandırma yollarını tmpfs üzerine yönlendirerek aştık.



Şekil 3. Bu grafik 2.812,8 saniyelik (46,9 dakika) toplam duvar saati süresinin farklı aşamalara dağılımını göstermektedir. Toplam sürenin yüzde 96,6 gibi büyük bir kısmı sadece generate aşamasına harcanırken engine boot, tokenisation ve teardown adımları çok küçük bir paya sahiptir.

10. Inference Strategy: Explainability Routine

Açıklanabilirlik tasarımında çözülmesi gereken asıl sorun, bağımsız bir zero-shot explainer yazmak değildi. Bizden istenen şey doğrudan predictor mimarisinin kendi gerekçesini sunmaktı. Bu talebi karşılamak için, modelin tahmin anında kullandığı karar değişkenlerini olduğu gibi dışa aktaran bir yapı kurduk. Servis mimarisini aynı imaj içinde çalışan yerel bir FastAPI ve ngrok tunnel ile sağladık. Motor tam olarak ayağa kalkana kadar /health uç noktası fail-closed mantığıyla çalışıyor ve gelen istekleri reddediyor.

Açıklanabilirlik rutini dört temel bileşene dayanıyor. İlk bileşen olan posterior, sonradan eklenmiş bir güven skoru değildir. Bu değer doğrudan argmax işleminin üzerinde alındığı forced-token üçlü vektörünün ta kendisidir. İkinci bileşen olan occlusion attribution mekanizmasında, her field sırayla boşaltılıyor ve satır aynı yoldan yeniden puanlanıyor. İki durum arasındaki log-posterior farkı, o field için doğrudan katkı değerini veriyor. Bu yöntem tek-feature coalition barındıran Shapley tarzı bir atıf sunuyor. Surrogate üzerinden kestirilmiyor. Tam aksine deploy edildiği haliyle model üzerinde kesin bir ölçüm yapıyor. Bu işlemin maliyeti field başına yalnızca tek bir max_tokens=1 prefill adımıdır.

Üçüncü bileşen olarak routing margin değerini hesapladık. Bu metrik en yüksek iki log-probability arasındaki farkı yansıtıyor ve router mekanizmasının sıralamada kullandığı sayıyı gösteriyor. Dördüncü nokta ise IPF mekanizmasının doğasıyla ilgilidir. IPF küresel bir işlem olduğu için tek satırda yeniden üretilemiyor. Bu kısıtlamadan dolayı arayüzde doğrudan ham posterior gösteriliyor ve bu durum kullanıcıya açıkça söyleniyor.

Geliştirdiğimiz arayüzün tipik kullanım akışı oldukça basittir. Kullanıcı ilk olarak bir row_id seçimi yapıyor. Ardından sistem ilgili ürün alanlarını ekrana getiriyor. Bu aşamada query alanı düzenlenebilir bir yapıda sunuluyor. Son adımda Explain aksiyonu tetikleniyor ve sistem tüm bu hesaplamaları canlı olarak yürütüp sonuçları ekrana döküyor.

11. Ne İşe Yaradı, Ne Yaramadı

Çözüm mimarimizde hem hızı hem de doğruluk payını artıran temel kararları netleştirdik. Üretken bir yaklaşım yerine forced-token read-out mekanizmasını kullandık. Bu sayede model doğrudan etiket marjinalini üretti. Tokenisation aşamasında uyguladığımız prefix-splice tokenisation işlemi süreyi 588,0 saniyeden 27,0 saniyeye indirdi. Donanım tarafında INT8 seçimi SM80 tensor core mimarisiyle doğal bir

uyum yakaladı. Champion modelinin attention backend katmanında FlashInfer kullanmak süreci belirgin şekilde hızlandırdı. Olasılıkları kalibre etmek için uyguladığımız IPF yönteminde ise lam=0,5 değeri en yüksek macro-F1 skorunu verdi.

Private test setinin 30.000, 30.000 ve 30.000 şeklinde dengeli dağıldığını biliyorduk. Çıktıyı bu dağılıma tam olarak sabitlemek cazipti ancak denedik ve vazgeçtik. Gerçek dağılıma tam oturmak, IPF lam=0,5 ile yarı oturmaktan 0,0067 daha kötü bir macro-F1 üretti. Cascade mimarisi de benzer bir kaderi paylaştı. Ucuz bir gate modeli kullanıp en zorlu örnekleri champion modele aktarmak teoride mantıklıydı. Pratikte cascade hızı artırdı fakat skoru düşürdü. Gate modeli tüm satırları puanlamak zorunda kaldığı için toplam sürenin büyük kısmını tüketti. Tek bir champion modelin 0,9131 skoruna ulaşmak için karmaşık bir cascade kurmanın getirdiği performans kaybı, kazanılan zamana değmedi.

Teknik altyapıda da bazı denemelerimiz başarısız oldu. Champion yapısında çok iyi çalışan FlashInfer kütüphanesini gate modeline zorlamak istedik. Gate modeline FlashInfer zorlamak sistemi 1,56x yavaşlattı. Tokenisation sürecini hızlandırmak amacıyla disk cache denedik ve reddettik. Diskten okuma ve hafızada yeniden kurma işlemi 3,92 saniye sürdü. Sıfırdan yeniden hesaplama ise yalnızca 1,55 saniye alıyordu. Disk cache kullanmak, veriyi yeniden hesaplamadan 2,5 kat daha yavaş olduğu için bu optimizasyon çabasından vazgeçtik.

En öğretici başarısızlığımız kural tabanlı ön-eleme turu oldu. Champion modeline gitmeden önce basit kurallarla sample elemeyi hedefleyen 10 agent içeren bir tur tasarladık. Tasarımın sonunda sıfır kural ship ettik. Bu bir efor sorunu değil, doğrudan bir mekanizma sorunuydu. Analizlerimizde router score ile gate confidence arasındaki korelasyonu -0,845 olarak ölçtük. Ucuz bir kuralın bulduğu yüksek-precision cebi, router'ın zaten reddettiği cepti. 18 farklı lexical predicate üzerinde precision ile lift birbirine zıt hareket ediyordu ve aralarındaki korelasyon -0,70 düzeyindeydi. Escalate edilen set içinde 0,95 eşiğinde sertifiye edilebilir sıfır satır bulduk. Gözlem sayısının 20 ve üzerinde olduğu 724 conjunction yapısının hiçbiri 0,90 sınırını geçemedi.

12. Ablation Study

Cascade mimarisinde gate oranını artırdıkça performansın nasıl değiştiğini ölçtük. Yönlendirme eşiklerini kademeli olarak gevşeterek sistemi test ettik. Elde ettiğimiz değerler aşağıdaki tabloda yer alıyor.

Yapılandırma	macro-F1	Süre (dk)
cascade @%5	0,8741	24,8
cascade @%10	0,8798	28,6
cascade @%15	0,8903	29,4
cascade @%20	<=0,8935	32,8
champion-only	0,9131	46,9

Yeni yönlendirilen 1.000 satır başına flip sayısı sırasıyla 375,6, 346,4 ve 274,0 olarak gerçekleşti. Her 5 puanlık adımda flip sayısı yaklaşık 0,79 kat azalıyor. Champion modelin tüm satırları işlemesi 46,9 dakika sürüyor ve en yüksek skoru veriyor. Bu yüzden cascade mimarisinden vazgeçtik.

Sınıf dağılımını bilinen oranlara zorlamak için farklı read-out parametrelerini denedik. Set dengeli olduğu için çıktıları bu dengeye yaklaştırmak istedik.

Read-out Yöntemi	macro-F1
argmax	0,8840
IPF lam=0,25	0,8874
IPF lam=0,50	0,8879
IPF lam=0,75	0,8860
IPF lam=1,00	0,8814
hard pin	0,8812

En iyi sonucu lam=0,50 seviyesinde elde ettik. Dağılımı gerçek oranlara tam olarak oturtmak, yarı oturtmaktan 0,0067 daha düşük bir performans üretiyor.

İşlem süresini 90.000 satır için ölçekleyerek tokenisation darboğazını inceledik. Varsayılan kütüphane ayarları çok sayıda thread kaynağını boşta bırakıyordu.

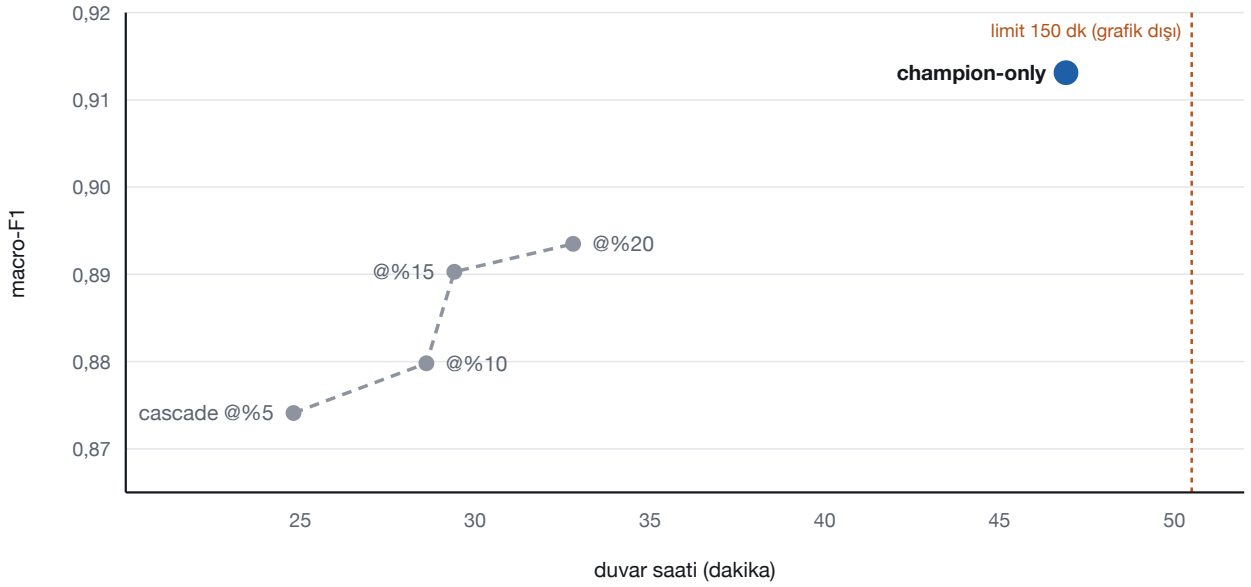
Tokenisation Yöntemi	Süre (sn)
serial loop	567,5
batch + PARALLELISM false	401,4
batch + parallel	16,5
spliced + batch + parallel	7,0

Çevresel değişkeni düzelterek thread kullanımını açmak tek başına 24 kat hızlanma sağladı. Paylaşılan metni bir kez işleyip gövdeye eklediğimiz son yöntem ile toplamda 81,1 kat hız artışı elde ettik.

Prompt uzunluğunun generate süresine etkisini 10.000 satırlık bloklar üzerinde test ettik. Maliyetin prompt uzunluğuyla değil, doğrudan gövde ile ölçeklendiğini gördük.

Prompt Bölümü	Maliyet (sn/token)
prefix	0,0136
body	0,662

Bir body token işlemek, bir prefix token işlemekten 49 kat daha maliyetli. Bu durum paylaşılan önbellek mekanizmasının beklenen sonucudur. Bu yüzden prompt boyutunu yarıya indirmek generate süresini %50 değil, yalnızca %17 düşürüyor.



Şekil 1. Bu grafik beş yapılandırmanın macro F1 ve wall clock düzlemindeki dağılımını gösterir. Kesikli çizgiyle birleştirilen cascade adımları en fazla 0,8935 skorunu üretir. Champion tek başına 150 dakikalık limitin çok altında kalarak zirveye çıktığı için routing ile sağlanan zaman tasarrufu gereksizdir.

13. Geliştirme Ortamı

Geliştirme ve test süreçlerimizi vast.ai üzerinde kiraladığımız çeşitli sunucularda yürüttük. Ağırlıklı olarak A100-SXM4-40GB donanımını kullandık. Farklı aşamalarda RTX 4090, RTX PRO 6000 ve H100 kutularını da denedik. Temel yazılım yığınımızı vLLM 0.28.0, torch 2.13.0+cu130 ve flashinfer-python 0.6.16.post3 oluşturdu. Tüm nihai ölçümleri tek bir A100-SXM4-40GB üzerinde gerçekleştirdik. Bu donanım organizatörün değerlendirme ortamındaki hedef kartıyla birebir eşleşmektedir.

Konteyner yapısı için base image olarak nvidia/cuda:13.0.0-devel-ubuntu24.04 sürümünü tercih ettik. Standart bir dağıtım için genellikle runtime varyantı kullanılır. Biz bilinçli olarak devel imajını seçtik. Triton kütüphanesinin çalışma zamanında ihtiyaç duyduğu JIT derleme süreci ptxas ve nvcc araçlarını gerektiriyor. Bu derleyiciler runtime imajında bulunmuyor. Geliştirme makinelerindeki host driver sürümümüz ise 580.126.20 olarak kaydedildi.

14. Nihai Yapılandırma: Champion-Only

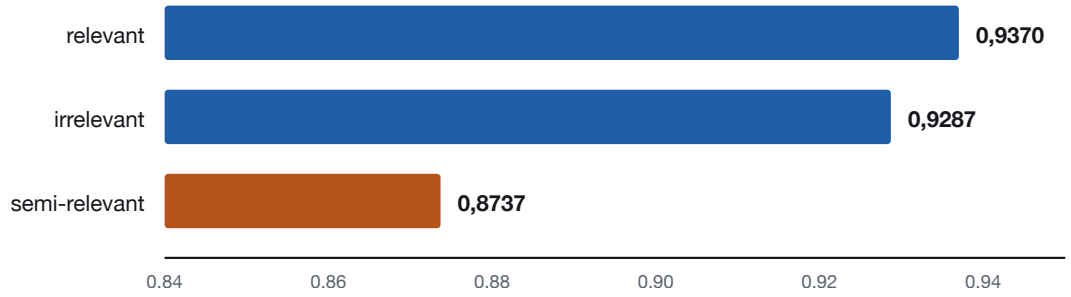
Nihai sistem mimarisi olarak champion yapılandırmasında karar kıldık. Geliştirme sürecinin büyük bir kısmında test ettiğimiz cascade yaklaşımı bir accuracy optimizasyonu değildir. Bu mekanizma temelde bir wall-clock optimizasyonudur. Denediğimiz her fraction değerinde cascade daha hızlı çalıştı ancak daha kötü sonuç verdi. Ölçümlerimize göre macro-F1 skorları **%5** için 0,8741, **%10** için 0,8798, **%15** için 0,8903 ve **%20** için en fazla 0,8935 seviyesinde kaldı. Buna karşılık champion yapılandırması 0,9131 değerine ulaştı. İki sistemin başa baş gelmesi ancak yaklaşık %30 routing oranında mümkün oluyor.

Organizatörün belirlediği süre sınırı 150 dakikadır. Champion yapılandırması 90.000 satırlık private test setini 46,9 dakikada tamamlıyor. Cascade mimarisinin bize kazandırdığı 17,5 dakikayla satın alınacak hiçbir ek avantaj bulunmuyor. Aksine, bu süre kazancı karşılığında 0,023 macro-F1 kaybediyoruz. Fraction parametresinin neden yanlış bir kontrol mekanizması olduğunu zaman ölçümleriyle de belirledik. Seçilen

oran ne kadar düşük olursa olsun gate modeli her fraction değerinde 90.000 satırın tamamını puanlamak zorundadır. Bu matematiksel zorunluluk nedeniyle cascade duvar saatinin %62 ila %80'i yalnızca gate işlemleriyle harcanıyor.

Tek modele dayalı bu yaklaşımın ikinci bir yapısal avantajı daha vardır. Champion yapılandırması MoE gate modelinin replicate kararsızlığını sisteme taşımaz. Yaptığımız ölçümlerde aynı arm'ın iki koşumu arasında %4,43 oranında label flip tespit ettik. Gate modeli kendi top-%10 zor setinin yalnızca 0,792'sini yeniden üretebiliyor. Champion sisteminde bir routing adımı bulunmadığı için bu rastgelelik tamamen ortadan kalkıyor.

Teslim ettiğimiz nihai sistem tek engine ve tek checkpoint üzerinden çalışıyor. Bu mimari 90.000 satırda 0,9131 macro-F1 skoruna 46,9 dakikada ulaştı. Sınıf bazında elde ettiğimiz F1 değerleri irrelevant için 0,9287, relevant için 0,9370 ve semi-relevant için 0,8737 olarak gerçekleşti. Performansın en zayıf halkası semi-relevant sınıfıdır. Bu durum beklenen bir topolojidir, çünkü semi-relevant sınıfı hem relevant hem de irrelevant bölgeleriyle sınır komşusudur ve karar yüzeyleri iki taraftan da baskı altındadır.



Şekil 2. Sadece champion yapılandırmasının 90.000 satır için sınıf bazında F1 değerleri gösterilmektedir. Relevant (0,9370) ve irrelevant (0,9287) kategorilerine kıyasla daha düşük skor alan semi-relevant (0,8737) sınıfı, diğer iki kategoriyle de sınır komşusu olduğu için en zayıf halkayı oluşturmaktadır.

Bu raporda geçen tüm sayılar tek bir NVIDIA A100-SXM4-40GB üzerinde ölçülmüştür. Cascade ve champion-only karşılaştırmaları aynı 90.000 satırlık değerlendirme seti, aynı read-out ve aynı donanım üzerinde alınmıştır. DreamTeam.